

Taller #1. Datasets, tokenización y embeddings

1. El objetivo de este ejercicio es analizar el comportamiento de dos estrategias de tokenización utilizadas en modelos de lenguaje modernos para tareas de Procesamiento del Lenguaje Natural (PLN): **WordPiece** y **SentencePiece**. Para ello, se utilizarán los datasets [Conll2002](#) y [Ancora](#), aplicando ambos tokenizadores sobre diferentes subconjuntos de entrenamiento, validación y prueba.

En el caso del dataset [Conll2002](#), se utilizarán directamente los conjuntos originales de entrenamiento (*train*), validación (*validation*) y prueba (*test*). Posteriormente, se tokenizarán únicamente las tres primeras sentencias de cada conjunto usando:

- El tokenizador **WordPiece** del modelo [BETO](#)..
- Un tokenizador basado en [SentencePiece](#), buscando una segmentación más cercana a palabras completas.

Para el dataset [Ancora](#), primero será necesario construir manualmente los conjuntos de entrenamiento, validación y prueba. Se deberá dividir el corpus de la siguiente manera:

- 70% para entrenamiento (*train*),
- 15% para validación (*validation*),
- 15% para prueba (*test*).

Una vez realizada la partición, se seleccionarán las tres primeras sentencias de cada conjunto y se aplicarán nuevamente los dos tokenizadores: WordPiece y [SentencePiece](#).

El propósito del ejercicio es comparar cómo ambos métodos fragmentan las palabras del español, observando especialmente:

- la cantidad de subpalabras generadas,
- la preservación de palabras completas,
- el tratamiento de palabras desconocidas,
- y las diferencias entre una tokenización orientada a subpalabras (WordPiece) y otra más cercana a unidades léxicas completas ([SentencePiece](#)).

Actividades propuestas

1. Cargar los datasets [Conll2002](#) y [Ancora](#).
2. Procesar los conjuntos de train, validación y testeo de [Ancora](#).
3. Mostrar ejemplos originales de las sentencias antes de tokenizar.
4. Aplicar el tokenizador WordPiece del modelo [BETO](#) a las sentencias de tokens de [Conll2002](#) y [Ancora](#).
5. Aplicar el tokenizador SentencePiece a las sentencias de tokens de [Conll2002](#) y [Ancora](#).
6. Comparar los resultados obtenidos.
7. Analizar cuál tokenizador conserva mejor la estructura léxica del español.

Ejemplo esperado de comparación

Sentencia original:

“El presidente visitó Barcelona durante la conferencia.”

Tokenización con Wordpiece (BETO)

['El', 'presidente', 'visitó', 'Bar', '##ce', '##lo', '##na', 'durante', 'la', 'conferencia', '.']

Tokenización SentencePiece

['_El', '_presidente', '_visitó', '_Barcelona', '_durante', '_la', '_conferencia', '.']

Nota importante. Se debe elaborar una conclusión donde se explique cuál de los dos enfoques produce una segmentación más adecuada para el español y cómo esto puede impactar el rendimiento de modelos de PLN basados en Transformers.

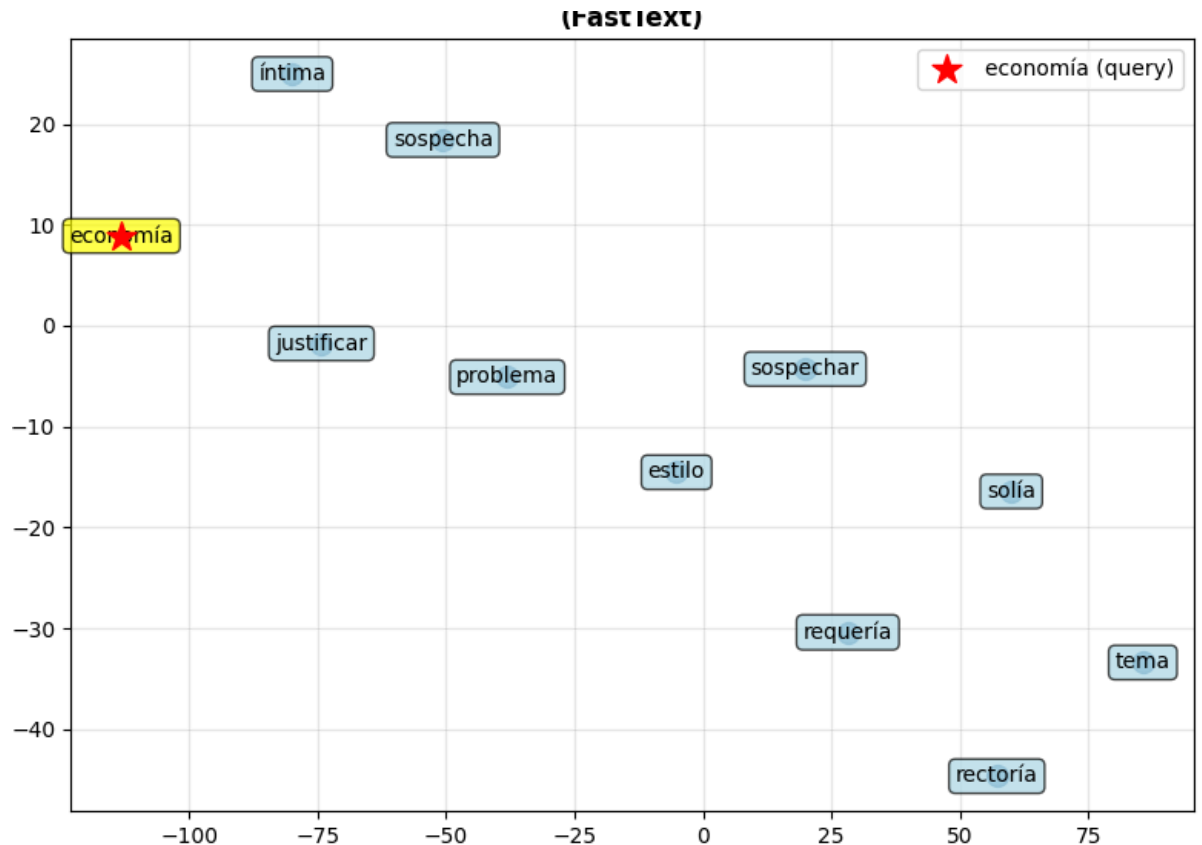
2. El objetivo de este ejercicio es estudiar la representación semántica de palabras usando Word2vec y Fasttext mediante las técnicas de reducción de dimensionalidad y visualización aplicadas a embeddings distribucionales obtenidos a partir del corpus [spanish word billion](#)

La práctica busca analizar cómo las palabras se relacionan semánticamente usando la similaridad distribucional y de qué manera pueden agruparse en espacios vectoriales utilizando métodos matemáticos como el Análisis de Componentes Principales (PCA) y t-Distributed Stochastic Neighbor Embedding (t-SNE).

Actividades propuestas

1. Cargar y preprocesar el dataset [spanish word billion](#) (usar streaming=True, ésta es una posibilidad para usar todo el dataset)
2. Preprocesar cada sentencia del dataset para que quede una lista de listas de palabras para usar [GENSIM](#). La idea es que éstas sentencias queden limpias de puntuaciones, palabras stop words, números y filtrado de tokens vacíos. ['raul', '']
3. Entrenamiento de los modelos **Word2Vec** y **Fasttext** con el embeddings de 300 y las sentencias del dataset.
4. Visualizar relaciones semánticas usando PCA y t-SNE para conjuntos de 100, 5000 y 10000 sentencias para los dos modelos de **Word2Vec** y **Fasttext**
5. Comparar el comportamiento de ambos algoritmos de reducción de dimensionalidad en términos de los conjuntos de sentencias y estructura global y local para Word2vec y Fasttext

6. Analizar vecindarios semánticos de palabras en español con respecto a los dos modelos, la idea es la de encontrar las 10 palabras más similares y visualizarlas en el plano cartesiano.



Nota: Se debe explicar cómo la calidad del chunking influye directamente en la precisión de los sistemas de recuperación semántica basados en embeddings, usando la ley de cosenos.

3. El objetivo de éste ejercicio es procesar documentos en formato [pdf](#), generar representaciones vectoriales (*embeddings*) de sus fragmentos textuales utilizando modelos de [Sentence Transformers de Hugging Face](#) y comparar el desempeño de distintos modelos en tareas de recuperación semántica de información mediante similitud coseno.

La práctica permitirá analizar cómo diferentes modelos de embeddings representan el significado semántico de los textos y cómo dichas representaciones impactan la calidad de la recuperación de información.

Actividades propuestas

1. Carga y preprocesamiento de los documentos en pdf:

Para la extracción de texto se debe utilizar [LangChain](#) junto con [pymupdf4llm](#) (recomendado por su mejor preservación del layout, estructura y contenido textual). El proceso debe cargar automáticamente todos los archivos PDF del directorio y extraer el contenido textual de cada documento.

2. Fragmentación del texto (Chunking)

Una vez extraído el contenido de los documentos, se debe realizar un proceso de fragmentación (*chunking*) del texto.

El objetivo es dividir cada documento en fragmentos de tamaño razonable que:

- mantengan coherencia semántica,
- preserven el significado contextual,
- y permitan una recuperación eficiente de información.

Se debe aplicar un fragmentador de texto apropiado, por ejemplo:

- `RecursiveCharacterTextSplitter`

La configuración del chunking debe justificarse adecuadamente, considerando parámetros como:

- tamaño del fragmento (*chunk_size=1000*),
- solapamiento (*chunk_overlap=200*),

3. Generación de embeddings de oraciones

Utilizar la biblioteca [Sentence Transformers](#) para generar representaciones vectoriales de cada fragmento textual obtenido en el paso anterior.

Se deben evaluar los siguientes modelos de embeddings:

Modelo	Dimensiones	Fortalezas
intfloat/multilingual-e5-base	768	Muy buen balance precisión/rendimiento
sentence-transformers/para phrase-multilingual-mpnet-base-v2	768	Similaridad semántica de alta calidad
BAAI/bge-m3	1024	Excelente retrieval semántico
sentence-transformers/para phrase-multilingual-MiniLM-L12-v2	384	Muy rápido y ligero

Para cada modelo se debe:

1. cargar el modelo correspondiente,
2. generar embeddings para todos los fragmentos,
3. almacenar las representaciones vectoriales resultantes,

4. Consulta de similitud semántica

Definir una oración de consulta, ya sea:

- ingresada por el usuario,
- o definida directamente en el código.

Para cada uno de los cuatro modelos:

1. generar el embedding de la consulta,
2. calcular la similitud coseno entre la consulta y el resto de fragmentos
3. identificar el fragmento más semánticamente similar.

5. Visualización en PCA y Tsne

Para cada modelo:

1. seleccionar:
 - la oración de consulta,
 - y el fragmento más similar recuperado mediante similitud coseno;
2. generar una gráfica en el plano cartesiano que muestre:
 - la posición de la oración de consulta,
 - y la posición del fragmento recuperado.

La visualización debe cumplir con las siguientes características:

- la consulta debe destacarse visualmente (por ejemplo, en rojo),
- el fragmento más similar debe representarse con otro color (azul o verde),
- cada gráfica debe incluir:
 - título,
 - leyenda,
 - etiquetas descriptivas,
 - y el nombre del modelo utilizado.

Finalmente, se debe interpretar visualmente:

- la cercanía entre embeddings,
- la separación semántica entre conceptos,
- y las diferencias observadas entre modelos de embeddings.